

The shift from credentials to identity

To understand why ‘secretless’ is the future, we must distinguish between credential-based and identity-based security. Traditional security relies on ‘something you know’—much like a physical door lock. If you have the key, you get in. The system does not care *who* you are, only that you possess the key.

In a typical CI/CD pipeline, the ‘key’ to your cloud environment is usually a static API password. This method has three major flaws: ‘implicit trust’, where the system cannot distinguish between a legitimate user and a thief using the same key; ‘permanence’, as keys often last for years, giving hackers an ‘infinite’ window of opportunity; and a heavy ‘management burden’, since rotating thousands of keys manually is prone to human error.

In 2026, the industry has shifted to ‘workload identity’. Instead of asking for a password, the system demands proof of identity. Think of this like airport security: you present a passport (identity) to receive a temporary boarding pass (access) valid only for your specific flight. The system verifies who you are before granting a short-lived permission to proceed.

Identity-based security is safer because it uses short-lived tokens that expire in minutes, making stolen tokens useless almost immediately. These tokens provide contextual proof, confirming exactly which repository and branch are requesting access. Since these tokens are created and destroyed automatically, there is no manual work, eliminating the risk of human error during secret rotation. By shifting to identity, we remove the ‘master key’ entirely, leaving no permanent secrets for attackers to steal.

Understanding the OIDC handshake

While the concept of identity-based security is straightforward, the actual

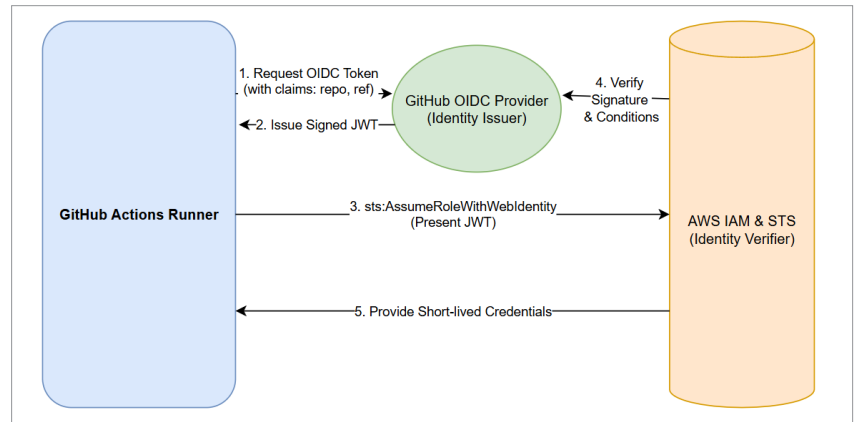


Figure 1: Technical flow

process relies on a precise cryptographic conversation known as the OIDC (OpenID Connect) handshake. This ‘secretless handshake’ between GitHub Actions and your cloud provider follows four distinct steps:

- 1. The request:** When the CI/CD pipeline begins, the GitHub runner requests a unique, short-lived OIDC token from GitHub’s identity provider. This token contains ‘claims’, which are verified statements about the job, such as the repository name and the specific branch being used.
- 2. The handshake:** GitHub sends this cryptographically signed token to your cloud provider (like AWS or Azure). This is not a password; it is a digital proof of identity that says, “I am a legitimate runner from this specific project.”
- 3. The verification:** The cloud provider receives the token and checks GitHub’s public keys to verify the digital signature. It then compares the ‘claims’ against a ‘trust relationship’ that is pre-configured. For example, it checks if it is instructed to trust tokens specifically from the sql-learning repository.
- 4. The reward:** Once the identity is verified, the cloud provider’s Security Token Service (STS) issues a temporary security token. This token has a very short lifespan and expires automatically the

moment your job finishes, ensuring that no permanent ‘master key’ is ever left behind.

Technical flow

The technical flow of a secretless architecture ensures that sensitive data never sits idle in a configuration file. The process begins inside the GitHub infrastructure, where the runner generates a cryptographically signed JSON Web Token (JWT). This token is unique to that specific job execution.

Once the token is sent to AWS, the IAM Identity Provider performs a real-time verification. It doesn’t just check if the token is valid; it inspects the ‘claims’ to ensure the request is coming from the authorised organisation (name of the GitHub organisation) and the correct repository (sql-learning). If the conditions match, AWS STS generates temporary credentials that expire automatically after the job. This flow, as shown in Figure 1, effectively reduces the ‘blast radius’ of a potential leak to near zero, as there are no long-term keys for an attacker to exploit.

Implementation guide

Implementing secretless architecture involves three primary steps: establishing trust in AWS, defining a secure role, and updating your GitHub workflow. By following these steps, you ensure that no permanent credentials ever leave your cloud environment.